



Università degli Studi di Salerno

Corso di Software Dependability

Rated Software Dependability Report



Data: 31/07/2026

Bruno Nesticò, Francesco Rao, Roberto Fiorenza

1. Introduction

1.1 Application Context

Rated is a web application designed to modernize and centralize the experience of rating and reviewing multimedia content, such as movies and TV series. The system operates within the digital entertainment services sector, offering an intuitive platform where enthusiasts can interact with a vast and constantly updated catalog. The primary objective is to provide users with an effective tool to express critical opinions while simultaneously offering administrators, catalog managers and moderators an advanced control panel tailored to their respective roles.

1.2 General System Description

Rated provides an integrated platform structured across two main fronts:

- **For users:** It allows them to explore the multimedia catalog, consult detailed technical data sheets, publish reviews, and assign star ratings to titles of interest. It also provides social features, such as rating other users' reviews and viewing various user profiles.
- **For administrators:** It provides a centralized interface enabling both types of designated administrators to perform their duties: Catalog management (insertion, modification, and removal of titles) and moderation of comments alongside user account control.

The project was developed following a three-tier architecture, which ensures a clear separation between the presentation layer (Frontend), the application logic (Backend), and persistence management (Database), thereby guaranteeing the software's maintainability and scalability.

1.3 Technologies Used

The system was built using the following technology stack:

- **Frontend:** HTML5, CSS3, and JavaScript (for creating a responsive and dynamic interface).
- **Backend:** Java, used for developing the business logic and implementing REST APIs.
- **Database:** MySQL, chosen for its reliability in managing complex relationships between users, titles, and reviews.

1.4 Project Documentation

The complete source code, UML diagrams, and detailed technical documentation are publicly available at the following GitHub repository:

<https://github.com/superfra01/Rated-Dependability/tree/main/Deliverables>

2. Github Workflow

2.1 Continuous Integration Workflow: CI

This workflow orchestrates the Continuous Integration (CI) pipeline, which automatically triggers on every push or pull request to the main branch. Its primary objective is to verify that all incoming repository changes successfully pass compilation, testing, quality assurance, and Docker image building.

The workflow consists of the following steps:

- **Repository Checkout:** The runner retrieves the current branch's codebase using the actions/checkout action.
- **Java Environment Setup:** The workflow configures JDK 17 via actions/setup-java. It enables Maven caching to accelerate subsequent builds by reusing pre-downloaded dependencies.
- **Compiling and Testing with Maven:** Running mvn verify executes the full build cycle for the application module. This step runs JUnit tests and generates coverage reports through JaCoCo, facilitating functional verification while tracking test coverage metrics.
- **Mutation Testing with PIT:** Utilizing the PIT plugin, the pipeline performs mutation analysis to evaluate how effectively the test suite detects artificial code defects. This technique is highly valuable for assessing software dependability, offering a far more robust measure of test suite quality than standard code coverage alone.
- **Upload Reports:** The workflow publishes JaCoCo and PIT reports as artifacts using actions/upload-artifact, ensuring they are readily available for the development team's offline review and analysis.
- **GitGuardian (Secrets Detection):** To mitigate the risk of exposing sensitive data, the workflow runs ggshield to scan the repository for leaked secrets, including API keys, tokens, and credentials.

- **Snyk (SCA and SAST):** The pipeline integrates Snyk to bolster software security through three key mechanisms:
 - **Snyk Open Source (SCA):** Scans Maven dependencies for known vulnerabilities, enforcing a strict minimum severity threshold.
 - **Snyk Monitor:** Continuously tracks dependency health over time, active on the main branch only.
 - **Snyk Code (SAST):** Performs static code analysis to detect security flaws while respecting the exclusions defined in the .snyk file.
- **SonarQube Analysis (Code Quality and Security):** Executed via the Maven plugin, this step publishes deep-dive analysis to SonarCloud, delivering insights into code smells, bugs, and vulnerabilities directly to the quality dashboard.
- **Build Docker Image:** As a final step, the pipeline builds the backend service's Docker image from the Dockerfile in the application module. Tagging the image with the unique commit hash ensures complete build traceability and reproducibility.

This workflow is available at the following address: <https://github.com/superfra01/Rated-Dependability/blob/main/.github/workflows/ci.yml>

2.2 Workflow: Docker Push to Hub

The Docker Push to Hub workflow drives the Continuous Delivery (CD) pipeline. Triggered either automatically on every push to the main branch or manually via workflow_dispatch, its objective is to build and publish the application's Docker image to Docker Hub, rendering it ready for deployment and distribution.

The main phases of the workflow are:

- **Code Checkout:** Pulls the repository codebase using actions/checkout.
- **JDK 17 Installation:** Replicates the Java environment setup from the CI workflow.
- **Application Build:** Compiles the executable artifact using mvn clean package -DskipTests. Tests are bypassed here because they are already validated during the CI pipeline.
- **Docker Buildx Setup:** Initializes a multi-architecture builder via docker/setup-buildx-action to generate images compatible across various hardware platforms.
- **Docker Hub Authentication:** Authenticates with Docker Hub using docker/login-action and credentials safely stored in GitHub Secrets, maintaining a secure separation between the codebase and sensitive credentials.

- **Build and Push the Image:** Leveraging docker/build-push-action, the workflow builds the Docker image for both linux/amd64 and linux/arm64 architectures, automatically publishing it to Docker Hub with the latest tag. It incorporates build caching (cache-from and cache-to) to optimize execution times.

This workflow is available at the following address: <https://github.com/superfra01/Rated-Dependability/blob/main/.github/workflows/docker-push.yml>

3. Formal Verification with OpenJML

3.1 Objectives and Context

OpenJML is a tool for the formal verification of Java programs annotated with JML (Java Modeling Language). It supports extended static checking (ESC), based on SMT solvers, and runtime assertion checking (RAC). OpenJML also provides JML specifications for part of the Java standard library, allowing application contracts to be checked together with the available library models.

OpenJML has been used to verify the system's core application logic, specifically catalog management, user interaction, review management, and content moderation. The formalization covers 27 Java files:

- **Model Classes:** FilmBean, FilmGenereBean, GenereBean, InteresseBean, PreferenzaBean, RecensioneBean, ReportBean, UtenteBean, ValutazioneBean, and VistoBean were annotated with class invariants and contracts for constructors, getters, and setters.
- **Data Access Objects:** contracts were introduced for the ten DAO classes, including FilmDAO, FilmGenereDAO, GenereDAO, InteresseDAO, PreferenzaDAO, RecensioneDAO, ReportDAO, UtenteDAO, ValutazioneDAO, and VistoDAO. These contracts specify preconditions on method parameters, frame conditions, nullability, and minimal properties of returned objects and collections.
- **Service Layer:** CatalogoService, RecensioniService, AuthenticationService, ModerationService, and ProfileService, were annotated to support modular reasoning about their interaction with domain objects and DAOs.
- **Utility Classes:** FieldValidator and PasswordUtility were also annotated. Their contracts describe validation and hashing properties, while their implementations are treated as trusted boundaries because they rely on external library APIs.

3.2 Application Strategy

The approach followed was bottom-up. The starting point was the Model layer, where invariants were introduced to capture essential local constraints, including non-negative identifiers and counters, non-null textual fields, and admissible numeric values.

For example, the invariants of `FilmBean` require a non-negative identifier, year, duration, and rating, together with non-null values for the film name, director, actors, and plot. Similar invariants are defined for users, reviews, reports, ratings, preferences, watched films, and interests.

Fields that can legitimately contain a null value, such as film posters and user icons, are marked explicitly with JML nullability annotations. This distinguishes intentionally nullable values from fields that must remain non-null to preserve a class invariant.

Constructors were specified to establish states satisfying the corresponding class invariants. Default constructors initialize every field, while parameterized constructors use preconditions to exclude arguments that would immediately violate an invariant. Constructor postconditions describe the resulting values of the initialized fields.

Constructors, pure getters, and setters were annotated to fully enforce and preserve these class invariants.

Once the Model layer had been specified, contracts were introduced for the DAO layer. DAO preconditions mainly concern non-null parameters, valid identifiers, and the availability of a non-null data source.

DAO read operations explicitly define the expected properties of their return values. For example, `FilmDAO.findById` ensures that the returned `FilmBean` matches the requested ID.

When a method returns a collection, it uses universal quantification (`\forall`) to guarantee that every item is non-null and matches the query parameters.

Because the persistent effects of MySQL operations cannot be represented directly through the Java heap model, DAO mutation operations use the broad frame condition:

`assignable \everything;`

This frame condition is applied to operations such as `save`, `update`, and `delete`. DAO read operations use:

`assignable \nothing;`

because they do not modify application-visible Java state.

JDBC-facing method bodies are excluded from extended static checking through `skipesc`. Ordinary methods excluded from ESC are also excluded from runtime assertion checking

through skiprac. The contracts of these methods remain available to callers during modular verification.

DAO constructors that perform infrastructure initialization use skipesc. In the OpenJML version adopted by the project, skiprac cannot be applied to constructor declarations, so it is used only on compatible ordinary methods.

After the Model and DAO layers had been specified, the Service Layer was analyzed using the contracts of the lower layers. This allows OpenJML to reason modularly about service methods without expanding the JDBC implementations invoked through the DAOs.

The Service Layer includes contracts for:

- catalog retrieval and film management;
- review retrieval, publication, deletion, voting, and reporting;
- authentication and registration;
- user-profile management;
- preferences, watched films, and watchlists;
- moderation operations.

3.3 Limitations and Issues Encountered

During the implementation of OpenJML, the main practical limitations concerned application boundaries involving external libraries, infrastructure services, relational persistence, and mutable Java collections.

3.3.1 Standard Library and Database Interaction

the absence of a model for the relational database and JDBC calls such as:

- `DriverManagerConnectionPool.getConnection();`
- `DataSource.getConnection();`
- `PreparedStatement.executeQuery();`
- `PreparedStatement.executeUpdate();`

are not represented as transformations of a logical MySQL database state.

For this reason, the corresponding JDBC method bodies are excluded using skipesc. Their contracts remain available for modular static reasoning, meaning that callers can rely on their preconditions and postconditions, although OpenJML does not directly verify the JDBC implementation against those contracts.

The corresponding ordinary methods also use `skiprac`, preventing runtime instrumentation of operations whose effects depend on database connections, SQL statements, result sets, and external persistent state.

Database-level properties, including referential integrity, uniqueness, foreign keys, primary keys, cascading deletions, and the validity of persisted relationships, remain the responsibility of the MySQL schema. Their behavior is also exercised through DAO and integration tests.

Other relevant external-library boundaries include:

- JNDI lookup operations used to obtain data sources;
- servlet-session invalidation;
- regular-expression matching used by `FieldValidator`;
- message-digest operations used by `PasswordUtility`;
- Base64 encoding used to represent password hashes.

The implementations of `FieldValidator` and `PasswordUtility` are marked with `skypesc` and `skiprac`, framing them as trusted boundaries having their behaviour verified with standard unit and integration tests.

3.3.2 Collections, Generics, Loops, and Quantifiers

Collections and generics are used extensively in DAO and Service methods. JML contracts on lists specify the properties of returned elements using universal quantification.

For example, a method returning reviews for a film can guarantee that:

- the returned list is non-null;
- every element is non-null;
- every review refers to the requested film identifier.

Similar contracts are used for films, genres, preferences, watched films, and user information.

The Service Layer uses imperative loops to process collections. Indexed loops are annotated, where useful, with:

- `loop_invariant`, describing properties preserved by every iteration;
- `decreases`, providing a termination argument.

Loop invariants describe bounds on indexes, relationships between indexes and collection sizes, and properties already established for processed elements. These

specifications allow OpenJML to reason about collection-processing methods without requiring an explicit expansion of every iteration.

Universal quantification through `\forall` is also used in method preconditions and postconditions. It expresses conditions such as the non-nullness of all elements in an input collection or the correspondence between every returned object and the parameters of the database query.

Validation properties requiring the presence of a character category are expressed through the negation of a universal condition, preserving the intended meaning while relying on `\forall`.

3.4 Verification Results

The complete ESC verification analyzes all 27 annotated Java files with OpenJML 21-0.23. The verification terminates successfully with exit code zero.

This result confirms that:

- all JML annotations are syntactically accepted;
- the specifications are type-correct;
- the proof obligations of methods not marked with `skypesc complete` without reported violations;
- constructors, getters, setters, invariants, service contracts, and supported loop specifications are mutually consistent under the declared preconditions.

A RAC compilation of the same 27 annotated files was also completed successfully, producing instrumented class files for runtime assertion checking.

Overall, with the OpenJML testing the correctness of the core business logic is effectively guaranteed.

4. Docker Environment Configuration

The application runs in a containerized environment that separates the Java web application from the relational database. This architecture improves portability and provides a consistent execution environment across different host platforms.

The application image is created through a multi-stage Dockerfile, which separates the Maven build phase from the Tomcat runtime environment. Docker Compose manages the complete deployment topology, consisting of the webapp service and the db_service service.

4.1 Application Dockerfile

The application image is built in two stages. The first stage uses the `maven:3.9.11-eclipse-temurin-17` image and is responsible for compiling and packaging the application. The `pom.xml` file is copied first, and Maven downloads the project dependencies through the dependency *go-offline goal*. The source code is then copied into the build environment, and the application is packaged as a WAR archive by running Maven.

The second stage uses the `tomcat:9.0.120-jdk17-temurin-jammy` image as the runtime environment. The default Tomcat web applications are removed, and curl is installed to support container health checks. A dedicated unprivileged tomcat user is created so that the application does not run with root privileges.

The `rated-app.war` archive generated during the build stage is copied into Tomcat's `webapps` directory and renamed to `ROOT.war`. This ensures the application serves directly from Tomcat's root context. The image exposes port 8080 and defines a health check that periodically verifies whether the application responds over HTTP. Tomcat is started through the standard `catalina.sh` run command.

Database parameters are not configured by a custom entrypoint script. Instead, Docker Compose defines the `CATALINA_OPTS` environment variable, which exposes the database settings as Java system properties. The JNDI data source descriptor is stored in `src/main/webapp/META-INF/context.xml` and is included in the WAR archive during the Maven packaging process. It uses the `DB_HOST`, `DB_PORT`, `DB_NAME`, `DB_USERNAME`, and `DB_PASSWORD` properties to construct the database connection.

The application Dockerfile is available at the following link:

<https://github.com/superfra01/Rated-Dependability/blob/main/Rated/Dockerfile>

4.2 Service Definition in Docker Compose

Container orchestration is defined in the `docker-compose.yml` file. The current configuration declares two services: `webapp` and `db_service`.

4.2.1 Database Service

The `db_service` service uses the `mysql:8.0.44` image as the application's relational database. Its configuration is supplied through environment variables.

Database files are stored in the named `db_data` volume, ensuring that data persists when the container is recreated. The project's `init.sql` file is mounted read-only in `/docker-entrypoint-initdb.d/` and is automatically executed by MySQL when the database volume is initialized for the first time.

The service does not publish MySQL port 3306 on the host. The database is accessible only to containers connected to `rated-network`, that include the `webapp`. This reduces unnecessary exposure of the database server.

A health check based on `mysqladmin ping` verifies that MySQL is ready to accept connections. Dependent services are started only after this health check succeeds. The service also uses the `unless-stopped` restart policy.

4.2.2 Application Service

The `webapp` service hosts the Java web application on Tomcat. Its image is always built locally from the project's `Dockerfile`. The build `.configuration` identifies the local build context, while `pull_policy build` instructs Docker Compose to build the image rather than pull a precompiled application image from a remote registry.

The service passes the database connection settings to Tomcat through `CATALINA_OPTS`. These settings identify `db_service` as the default database host and port 3306 as the internal database port. The corresponding Java system properties are used by the JNDI resource defined in the application's `META-INF/context.xml` file.

Through `depends_on` and the `service_healthy` condition, the `webapp` container waits for `db_service` to become healthy before starting. Tomcat's internal port 8080 is published on a configurable host port, which defaults to 8080 through the `APP_PORT` variable. The service uses the `unless-stopped` restart policy and communicates with the database through the `rated-network`.

The application's `docker-compose.yml` file can be found at the following link:

<https://github.com/superfra01/Rated-Dependability/blob/main/Rated/docker-compose.yml>

5. Testing Activities

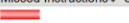
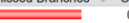
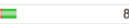
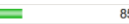
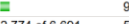
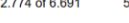
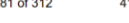
5.1 Code Coverage Analysis with JaCoCo

A code coverage analysis, on the preexisting test suite of unit and integration tests was performed using the JaCoCo plugin within the Maven build.

To ensure the reliability of the components, a minimum threshold of 80% code and branch coverage was established. This target is the result of a balanced trade-off as it is high enough to ensure that most execution paths and logical branches are effectively exercised, minimizing regression risks, but also realistic enough to avoid the creation of redundant "filler" tests written solely to inflate the percentage.

Finally, recognizing that high coverage metrics do not automatically equate to effective testing, these structural benchmarks were systematically complemented by checks on test quality and the actual meaningfulness of the executed scenarios.

Rated

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods	Missed	Classes
sottosistemi.Gestione_Utenti.view		0%		0%	84	84	298	298	43	43	12	12
sottosistemi.Gestione_Catalogo.view		0%		0%	53	53	172	172	31	31	8	8
model.DAO		84%		71%	46	121	173	839	16	69	0	10
sottosistemi.Gestione_Recensioni.view		0%		0%	24	24	77	77	20	20	5	5
model.Entity		76%		n/a	9	102	55	245	9	102	0	10
model		0%		0%	10	10	39	39	8	8	2	2
utilities		43%		0%	9	10	13	21	6	7	1	2
sottosistemi.Gestione_Recensioni.service		88%		85%	5	26	10	105	1	12	0	1
sottosistemi.Gestione_Utenti.service		90%		90%	6	42	15	126	3	27	0	3
sottosistemi.Gestione_Catalogo.service		93%		100%	1	17	5	70	1	14	0	1
Total	2,774 of 6,691	58%	181 of 312	41%	247	489	857	1,992	138	333	28	54

Created with JaCoCo 0.8.11.202310140853

Figure 5.1: JaCoCo report before updating test cases.

As shown in Figure 5.1, the initial test suite achieved: **58% instruction coverage**, with 2,774 missed instructions out of a total of 6,691; and **41% branch coverage**, with 181 missed branches out of a total of 312. The report also highlighted 857 missed lines out of 1,992. This data clearly indicated the presence of significant portions of the codebase that were never tested, particularly within the View packages (sottosistemi.Gestione_Utenti.view, sottosistemi.Gestione_Catalogo.view, sottosistemi.Gestione_Recensioni.view) and the utilities package.

Based on JaCoCo's initial report, a white-box analysis was conducted to pinpoint the test suite's weaknesses. The objective was to apply branch coverage to exercise unvisited decision paths, verify unconsidered edge cases, and test alternative paths related to input validation and exception handling.

Following these findings, the unit tests for the View were added and the Service and Utility components were heavily extended and refined. These updates significantly increased the coverage of branches.

Rated

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed Cxty	Missed Lines	Missed Methods	Missed Classes
sottosistemi.Gestione_Utenti.view		92%		90%	19 142	46 559	2 56	0 12
sottosistemi.Gestione_Catalogo.view		94%		97%	4 98	27 376	1 39	0 8
model.DAO		98%		91%	6 97	11 844	1 69	0 10
sottosistemi.Gestione_Recensioni.view		95%		100%	0 63	12 205	0 20	0 5
sottosistemi.Gestione_Recensioni.service		98%		93%	4 41	0 117	0 12	0 1
sottosistemi.Gestione_Utenti.service		98%		90%	5 52	1 143	0 27	0 3
model.Entity		100%		n/a	0 102	0 245	0 102	0 10
sottosistemi.Gestione_Catalogo.service		100%		94%	1 23	0 76	0 14	0 1
utilities		100%		100%	0 10	0 21	0 7	0 2
model		100%		75%	2 7	0 24	0 3	0 1
Total	298 of 8,510	96%	37 of 572	93%	41 635	97 2,610	4 349	0 53

Figure 5.2: JaCoCo report after updating test cases.

As shown in Figure 5.2, the improvements allowed the project to reach **96% instruction coverage** (only 298 missed instructions) and **93% branch coverage**, successfully meeting the minimum threshold set for the core system.

5.2 Mutation Testing Campaign with PIT

To evaluate the test suite’s actual ability to detect defects a mutation testing campaign was conducted using PIT.

5.2.1 Initial Analysis

Pit Test Coverage Report

Project Summary

Number of Classes	Line Coverage	Mutation Coverage	Test Strength
54	59% 1185/2012	47% 369/784	75% 369/494

Breakdown by Package

Name	Number of Classes	Line Coverage	Mutation Coverage	Test Strength
model	2	0% 0/39	0% 0/8	100% 0/0
model.DAO	10	84% 716/855	58% 194/332	68% 194/284
model.Entity	10	78% 190/245	86% 37/43	86% 37/43
sottosistemi.Gestione_Catalogo.service	1	93% 65/70	50% 19/38	50% 19/38
sottosistemi.Gestione_Catalogo.view	8	0% 0/174	0% 0/57	100% 0/0
sottosistemi.Gestione_Recensioni.service	1	90% 95/105	84% 59/70	95% 59/62
sottosistemi.Gestione_Recensioni.view	5	0% 0/77	0% 0/22	100% 0/0
sottosistemi.Gestione_Utenti.service	3	88% 111/126	89% 57/64	92% 57/62
sottosistemi.Gestione_Utenti.view	12	0% 0/300	0% 0/133	100% 0/0
utilities	2	38% 8/21	18% 3/17	60% 3/5

Figure 5.3: Pit Test Coverage Report Before changes.

In the first run, PIT was configured to analyze the entire project ecosystem. The summary of the initial results (Figure 5.3) yielded the following project-level metrics:

- **Line Coverage:** 59% (1,185 covered lines out of 2,012)
- **Mutation Coverage:** 47% (369 mutants killed out of 784)
- **Test Strength:** 75% (369 mutants killed out of 494)

These results demonstrated that, despite having baseline structural coverage, the test suite allowed a significant number of mutations to survive at critical domain points. Starting from the list of surviving mutants, new tests were engineered to cover previously unconsidered combinations of conditions, to verify return values and side effects more accurately, and to tighten existing assertions.

5.2.2 Final Analysis

Pit Test Coverage Report

Project Summary

Number of Classes	Line Coverage	Mutation Coverage	Test Strength
53	99% 2558/2590	98% 976/999	98% 976/997

Breakdown by Package

Name	Number of Classes	Line Coverage	Mutation Coverage	Test Strength
model	1	95% 20/21	100% 8/8	100% 8/8
model.DAO	10	99% 825/826	100% 298/298	100% 298/298
model.Entity	10	100% 245/245	100% 43/43	100% 43/43
sottosistemi.Gestione_Catalogo.service	1	100% 75/75	100% 44/44	100% 44/44
sottosistemi.Gestione_Catalogo.view	8	97% 365/378	95% 129/136	96% 129/134
sottosistemi.Gestione_Recensioni.service	1	100% 117/117	98% 83/85	98% 83/85
sottosistemi.Gestione_Recensioni.view	5	99% 202/205	96% 81/84	96% 81/84
sottosistemi.Gestione_Utenti.service	3	100% 141/141	96% 74/77	96% 74/77
sottosistemi.Gestione_Utenti.view	12	98% 547/561	96% 199/207	96% 199/207
utilities	2	100% 21/21	100% 17/17	100% 17/17

Figure 5.4: Pit Test Coverage Report After changes.

PIT was subsequently rerun following the test suite enhancements. The final results (Figure 5.4) showcase a substantial enhancement:

- **Line Coverage:** 99% (2558/2590)
- **Mutation Coverage:** 98% (976/999)
- **Test Strength:** 98% (976/997)

The package breakdown highlights excellent progress in previously weak areas. For example:

- **sottosistemi.Gestione_Utenti.view:** Mutation coverage jumped from 0% to 96% (199/207 mutants killed), with test strength hitting 96%.
- **sottosistemi.Gestione_Catalogo.view:** Mutation coverage rose from 0% to 95% (129/136 mutants killed).
- **model.DAO:** Mutation coverage improved from 58% to 100%, solidifying the reliability of data access operations.

5.3 Performance Testing

This section details the performance testing activities carried out to evaluate the system's efficiency and measure the effect of applied optimizations.

To ensure strict comparability across all measurements, executions were performed while keeping the operating conditions constant, matching the execution environment defined in Section 1.3: **Lenovo laptop with AMD Ryzen 5 5500U, 8 GB RAM, Windows 11 (24H2), JDK 17, and MySQL 9.2.0.**

A previous analysis and system enhancement was done on the project for the Software Sustainability exam.

Link:

<https://github.com/superfra01/Rated-Dependability/tree/main/Deliverables/2.Deliverables-Sustainability>

After that exam the functions offered by our platform for the Software Engineering Advanced Techniques exam have been expanded.

Link:

<https://github.com/superfra01/Rated-Dependability/tree/main/Deliverables/3.Deliverables-ISTA>

Because of this it has been decided to redo the analysis and enhancement process using JMH and SonarQube with creedengo rules.

5.3.1 JMH Microbenchmarks: Configuration and Measurement Criteria

The JMH campaign was designed to measure the average time per operation of service-layer methods using Mode.AverageTime and the ns/op unit. The adopted configuration was:

- **Benchmark mode:** Mode.AverageTime
- **Time unit:** NANOSECONDS (ns/op)

- **Warmup:** 15 iterations of 1 second
- **Measurement:** 40 iterations of 1 second
- **Fork:** 2

The warmup choice of 15 iterations was selected after testing several combinations; it proved to be the optimal value for stabilizing the AverageTime result, effectively reducing variability caused by typical JVM factors (JIT compilation, profiling, and caching).

Benchmarks were executed on core application components relevant to the Rated ecosystem, including: CatalogoService, RecensioneService, AuthenticationService, and application utilities. All results were exported to txt and versioned in the repository, separated into before and after directories:

- **Before:**

<https://github.com/superfra01/Rated-Dependability/tree/main/Deliverables/4.Deliverables-Dependability/Analisi/JMH/before>

- **After:**

<https://github.com/superfra01/Rated-Dependability/tree/main/Deliverables/4.Deliverables-Dependability/Analisi/JMH/after>

5.3.2 Static Analysis with Creedengo

Prior to refactoring the code for JMH optimizations, a static analysis was conducted with SonarQube Community, extended via the **Creedengo** ruleset. In this context, Creedengo's goal is not merely "style cleanup," but rather to detect patterns that introduce resource waste, thereby increasing execution time and system load.

▼ Rule

Search for rules...		
(Java) Variable can be made constant	140	
(Java) Don't use the query SELECT * FROM	18	
(Java) Use ++i instead of i++	3	
(Java) Avoid getting the size of the collection in t...	2	
(Java) Do not call a function when declaring a for...	2	
(Java) Avoid usage of static collections.	1	

6 shown

Figure 5.5: Creedengo rules and detected issue count (initial phase).

The initial static analysis identified six distinct categories of issues, totaling 166 violations directly associated with Creedengo rules. The detected categories comprised:

- **(Java) Variable can be made constant (140 issues):** This was the most frequent violation, representing the vast majority of cases (approximately 84%). It recommends declaring variables that are never reassigned as final. While modern JIT compiler optimizations may limit the immediate performance impact, systematically neglecting this practice increases the overall energy footprint and can gradually degrade code efficiency over time.
- **(Java) Don't use the query SELECT * FROM (18 issues):** This represents the most critical violation from both a sustainability and efficiency perspective. Querying with SELECT * retrieves unnecessary columns, which needlessly inflates database disk I/O, increases network data transfer, and elevates application memory consumption.
- **(Java) Avoid usage of static collections (1 issue):** Although occurring only once, this violation has a substantial impact on memory management. Static collections persist in memory throughout the entire application lifecycle, continuously consuming energy solely to maintain their persistent state.
- **Loop and Micro-optimizations (7 issues combined):** The remaining issues involved loop inefficiencies, specifically: using ++i instead of i++, avoiding the evaluation of collection size within loop conditions, and avoiding function calls inside for loop declarations. These anti-patterns waste redundant CPU cycles and energy during repeated executions.

The refactoring intervention was strictly driven by the impact of the findings:

- **High Priority:** Rules with a direct impact on database I/O, network traffic, CPU cycles, and memory management (SELECT *, static collections, and loop definitions).
- **Low Priority:** Stylistic or micro-optimization rules (final modifiers).

Here is a breakdown of the technical fixes applied:

- **Fix for SELECT * FROM (High priority):** All offending queries were completely rewritten by explicitly listing only the required fields. This drastically reduced the payload of data transferred from the database.

```

    Rated/src/main/java/model/DAO/FilmDAO.java
    @@ -55,7 +55,7 @@ public void save(final FilmBean film) { // Parametro final
    55     }
    56
    57     public FilmBean findById(final int idFilm) { // Parametro final
    58 -     final String query = "SELECT * FROM Film WHERE ID_Film = ?";
    58 +     final String query = "SELECT ID_Film, Locandina, Nome, Anno, Durata, Generi, Regista, Trama, Valutazione, Attori FROM Film WHERE ID_Film = ?";
    59
    59     try (final Connection connection = dataSource.getConnection();
    60         final PreparedStatement ps = connection.prepareStatement(query)) {
    61         ps.setInt(1, idFilm);
    61 +
    62     }
    62
    
```

- **Fix for static collections (High priority):** The collection was converted into an instance or local variable, ensuring that the allocated memory is properly released by the Garbage Collector at the end of the HTTP request or specific operation.

```

    2 files changed +1 -1 lines changed
    Rated/src/main/java/model/DriverManagerConnectionPool.java
    @@ -8,7 +8,7 @@
    8
    9     public class DriverManagerConnectionPool {
    10         // Reso final perché inizializzato inline e mai riassegnato (solo modificato il contenuto)
    11 -     private static final List<Connection> freeDbConnections = new LinkedList<Connection>();
    11 +     private static List<Connection> freeDbConnections = new LinkedList<Connection>();
    12
    13 +     static {
    14         try {
    
```

- **Fix for Loop Optimizations (High priority):** Invariant function calls and `.size()` evaluations were extracted and moved outside the loop declarations, ensuring they are computed only once and reused.
- **Fix for Variable can be made constant (Low priority):** Leveraging the IDE's automated refactoring capabilities, the `final` modifier was systematically added to all variables and parameters that are not modified after their initialization.

```

    - public void doPost(HttpServletRequest request, HttpServletResponse response) throws ServletException, IOException {
    35 + public void doPost(final HttpServletRequest request, final HttpServletResponse response) throws ServletException, IOException { // Parametri final
    36
    36 -     HttpSession session = request.getSession(true);
    37 -     UtenteBean user = (UtenteBean) session.getAttribute("user");
    37 + final HttpSession session = request.getSession(true); // Locale final
    38 + final UtenteBean user = (UtenteBean) session.getAttribute("user"); // Locale final
    39 + if(user.getTipoUtente().equals("GESTORE")) {
    40
    40 -         int anno = Integer.parseInt(request.getParameter("annoFilm"));
    41 -         String Attori = request.getParameter("attoriFilm");
    42 -         int durata = Integer.parseInt(request.getParameter("durataFilm"));
    43 -         String Generi = request.getParameter("generiFilm");
    44 -         String Nome = request.getParameter("nomeFilm");
    45 -         String Regista = request.getParameter("registaFilm");
    46 -         String Trama = request.getParameter("tramaFilm");
    41 + final int anno = Integer.parseInt(request.getParameter("annoFilm")); // Locale final
    42 + final String Attori = request.getParameter("attoriFilm"); // Locale final
    43 + final int durata = Integer.parseInt(request.getParameter("durataFilm")); // Locale final
    44 + final String Generi = request.getParameter("generiFilm"); // Locale final
    45 + final String Nome = request.getParameter("nomeFilm"); // Locale final
    46 + final String Regista = request.getParameter("registaFilm"); // Locale final
    47 + final String Trama = request.getParameter("tramaFilm"); // Locale final
    
```

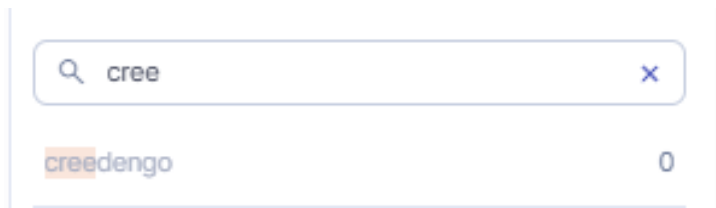


Figure 5.6: SonarQube re-run after fixes.

Following the refactoring interventions, the static analysis was re-run with SonarQube using the same Quality Profile. The result, as highlighted in Figure 5.6, confirms the total resolution (**0 issues**) of all critical and non-critical violations marked with the Creedengo tag.

5.3.3 Validation: Re-running JMH and Before/After Comparison

Once the Creedengo-driven refactoring was applied, the JMH suite was re-executed using the identical runtime configuration to guarantee a meaningful before-and-after comparison.

The following table presents an excerpt from the **ProfileService** benchmarks, demonstrating a clear reduction in the average time per operation across multiple scenarios:

Table 5.1: JMH comparison (ns/op) for ProfileService benchmarks.

Benchmark	Before (ns/op)	After (ns/op)	Reduction %
benchmarkAggiungiAllaWatchlist	4540,797	3607,234	20.56%
benchmarkPasswordUpdate	9982,091	7716,780	22.69%
benchmarkProfileUpdate	14696,992	11558,612	21.35%

Note: The table above highlights a sample scenario. The repository contains the full suite of measurements for other services with complete txt files linked at the beginning of this section.

Ultimately, the testing and optimization chain (JMH → Creedengo → Fixes → JMH re-run) validates the hypothesis that eliminating performance-related anti-patterns significantly reduces computational waste and boosts the performance of the Rated application.

6. Security Mechanism Integrated in GitHub

Alongside the CI/CD pipeline, which already integrates specialized security tools like GitGuardian, Snyk, and SonarQube, the Rated repository leverages GitHub's native security features to implement a comprehensive shift left security model. This multilayered approach aims to identify and mitigate security risks at the earliest possible stage, stopping vulnerabilities before the code is ever released.

The enabled native capabilities focus on three critical areas:

- **Supply Chain Security (Dependency Graph and Dependabot):** The system automatically analyzes project dependencies and continuously cross references them against up to date vulnerability databases. Beyond simply flagging risks, Dependabot simplifies maintenance by automatically generating corrective Pull Requests, which can also be delivered in grouped formats. This significantly reduces the likelihood of incorporating compromised libraries and accelerates response times to external security events.
- **Secret Protection (Secret Scanning and Push Protection):** A proactive scanning process is active to detect patterns matching sensitive credentials, including tokens, API keys, and private secrets. Push protection acts as a preventive gatekeeper, automatically blocking push operations if any recognized secrets are detected within the code. This mechanism prevents accidental credential leaks from ever entering the repository's git history, avoiding potential compromises and the need for stressful revocation and rotation procedures.
- **Vulnerability Reporting Management (Private Vulnerability Reporting):** A dedicated, secure channel is provided for users and researchers to privately report potential security concerns. This mechanism establishes a responsible disclosure framework, giving the Rated development team the opportunity to analyze and resolve vulnerabilities confidentially before they are publicly disclosed.

7. Security Analysis with GitGuardian, Snyk and Sonarqube

To enhance system quality from a security standpoint, a multi-layered analysis strategy was adopted utilizing complementary tools: SonarQube for static code analysis and vulnerability detection, GitGuardian to identify secrets, tokens, or credentials accidentally exposed in the repository, and Snyk for deep code-level security analysis to detect vulnerable patterns.

The objective for this phase was defined in an explicit and measurable way: to achieve zero security vulnerabilities across all adopted tools. The following sections describe the detected issues, their potential impact, and the corrective actions applied to reach this target.

7.1 SonarQube: fixing security issues

In the initial run, SonarQube flagged 97 security issues. Specifically, 88 findings were attributable to unhandled exceptions in servlet methods, primarily when executing forward calls without explicitly handling potentially thrown exceptions, specifically ServletException and IOException.

Without explicit handling, an error during request forwarding can lead to uncontrolled system behavior, such as flow interruptions, inconsistent state responses, or the accidental exposure of detailed error stack traces, thereby compromising overall robustness.

The remaining 9 findings (7 in JavaScript and 2 in Java) warned about hardcoded credentials.

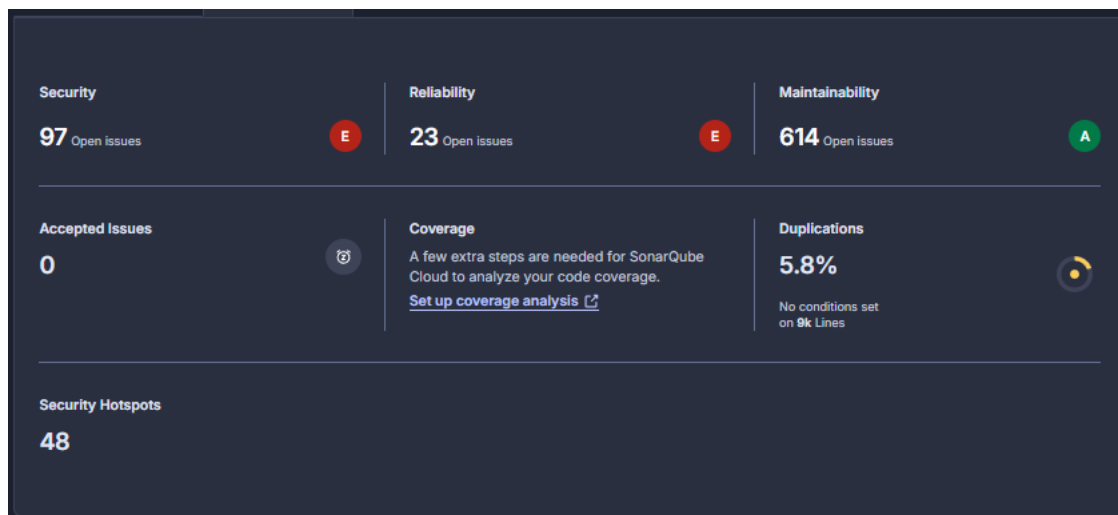


Figure 7.1: SonarQube 97 security issues detected.

Applied Fixes

To resolve the vulnerabilities, all forward calls and sensitive servlet operations were encapsulated within try/catch blocks to intercept ServletException and IOException, enabling the system to handle errors in a controlled and secure manner, while the 9 credential warnings were audited and confirmed as false positives. After applying these

fixes, SonarQube was re-run and the target was successfully reached, with the dashboard reporting zero security vulnerabilities.

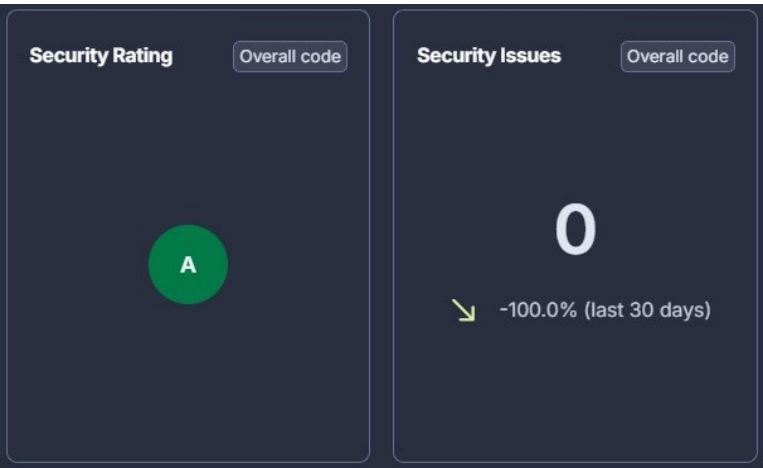


Figure 7.2: SonarQube 0 security issues detected after fixes.

7.2 GitGuardian

The analysis performed by GitGuardian highlighted four findings. A manual review confirmed that these were false positives. The first two involved dummy credentials used strictly for testing activities, while the fourth was associated with benchmarking scripts and held no security risk. The third finding, as shown in the image below, was also a false positive flagging a configuration line in the docker-compose.yml file.

☐	Occurred date	Validity	Secret	Severity	Info	Tags	Status
☐	Jul 14th 23:58	No checker	Username Password	High	superfra01/Rated-Dependability Rated/src/test/java/unit/test_model/UtenteBeanTest... superfra01 151184247+superfra01@users.noreply.github.com	Test file Default branch	Ignored Not a secret (false positive)
☐	Jul 14th 23:58	No checker	Generic Password	High	superfra01/Rated-Dependability Rated/src/test/java/unit/test_model/FieldValidatorTe... superfra01 151184247+superfra01@users.noreply.github.com	Test file Default branch	Ignored Not a secret (false positive)
☐	Jul 14th 15:08	No checker	Generic Password	High	superfra01/Rated-Dependability Rated/docker-compose.yml superfra01 151184247+superfra01@users.noreply.github.com	Default branch	Ignored Not a secret (false positive)
☐	Feb 23rd 16:02	No checker	Generic Password	High	superfra01/Rated-Dependability Rated/benchmark/AuthenticationServiceBenchmark... superfra01 +1 151184247+superfra01@users.noreply.github.com	From historical scan Default branch	Ignored Not a secret (false positive)

Figure 7.3: GitGuardian scan results

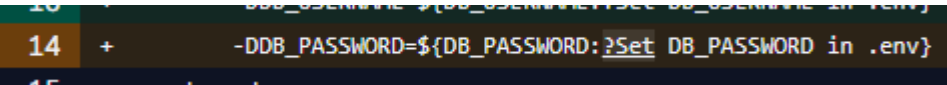


Figure 7.4: Highlight of docker-compose.yml finding

`-DDB_PASSWORD=${DB_PASSWORD:?Set DB_PASSWORD in .env}`

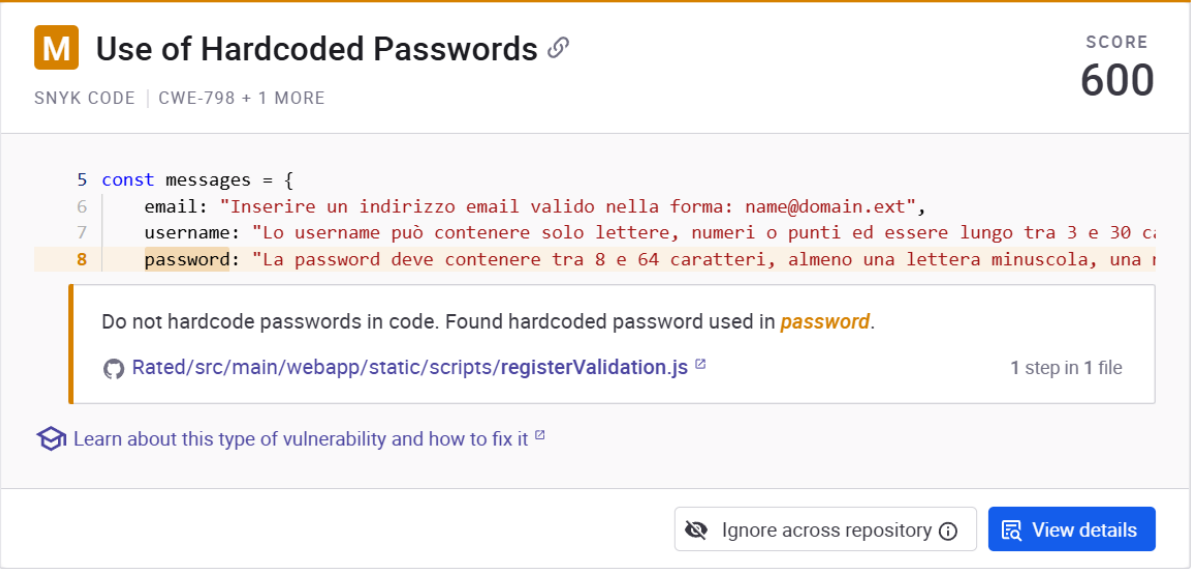
This line does not contain any hardcoded credentials. Instead, it utilizes Bash and Docker Compose parameter expansion syntax. The expression `${DB_PASSWORD:?Set DB_PASSWORD in .env}` acts as a validation check, meaning that if the environment variable `DB_PASSWORD` is null or unset, execution halts and the specified error message is printed.

Since no actual password is leaked and only a reference to a variable is used, this is a clear false positive triggered solely by the scanner detecting the keyword `PASSWORD`.

7.3 Snyk: resolving security issues (Snyk Code Analysis)

For code-level security analysis, Snyk Code was employed to perform static application security testing. The goal was once again to achieve zero issues, eliminating any potentially vulnerable patterns identified in the initial run.

The analysis found only a single false positive, triggered by the scanner detecting the word `password` in a non-sensitive context.



The screenshot displays a Snyk Code alert titled "Use of Hardcoded Passwords" with a score of 600. It identifies a hardcoded password in a JavaScript file. The code snippet shows a configuration object with fields for email, username, and password. The password field is highlighted as the source of the issue. A message box explains that passwords should not be hardcoded and provides the file path where the issue was found. At the bottom, there are links to learn more about the vulnerability and buttons to ignore the issue or view details.

M Use of Hardcoded Passwords [🔗](#) SCORE 600

SNYK CODE | CWE-798 + 1 MORE

```
5 const messages = {
6   email: "Inserire un indirizzo email valido nella forma: name@domain.ext",
7   username: "Lo username può contenere solo lettere, numeri o punti ed essere lungo tra 3 e 30 c
8   password: "La password deve contenere tra 8 e 64 caratteri, almeno una lettera minuscola, una i
```

Do not hardcode passwords in code. Found hardcoded password used in **password**.

[🔗](#) Rated/src/main/webapp/static/scripts/registerValidation.js [🔗](#) 1 step in 1 file

[🎓 Learn about this type of vulnerability and how to fix it](#) [🔗](#)

[🚫 Ignore across repository](#) [🔍 View details](#)

7.4 Final Results

All security goals set for this phase were successfully achieved:

- **SonarQube:** SonarQube: Reduced from 97 security issues to zero by resolving unhandled exceptions in servlet forward calls using try/catch blocks and confirming the remaining 9 credential alerts as false positives.
- **GitGuardian:** All four hardcoded password alerts were manually reviewed and verified as false positives.

- **Snyk:** The single identified hardcoded password issue was manually verified as a false positive.

8. Final Considerations

The high standard achieved for the Rated project demonstrates that dependability is the direct result of a systematic, repeatable, and highly measurable engineering workflow.

Implementing a structured CI/CD pipeline, maintaining a rigorous focus on code quality, and embedding automated security checks turned dependability into a verifiable and noteworthy target.

Every optimized metric, resolved vulnerability, and automated validation step reinforces the application's stability, even when handling edge cases. Thanks to these activities, the Rated architecture is now significantly more robust and maintainable.